

Rootwell — Developer Guide

Practical, task-oriented reference for working in this repo. For *why* the system is shaped this way, see [docs/ARCHITECTURE.md](#). For *why each dependency was chosen*, see [docs/TECHNICAL_GUIDE.md](#).

1. Prerequisites

- Docker + Docker Compose (v2 CLI — `docker compose`, not `docker-compose`). Everything runs in containers; you do **not** need a local JDK, Maven, or Python interpreter installed to build or run any service — every Dockerfile is a self-contained multi-stage build.
- Nothing else is required to run the app in mock mode. Real credentials (`ANTHROPIC_API_KEY`, `RESEND_API_KEY`) are optional — see §5.

2. Running it

```
docker compose -f infra/docker-compose.yml up -d --build
docker compose -f infra/docker-compose.yml ps           # everything should be Up
```

- Frontend: `http://localhost:3000`
- Gateway (for curling the API directly): `http://localhost:8082`
- Redis (host-mapped, for `redis-cli` debugging): `redis-cli -p 6380`

Rebuild a single service after editing it:

```
docker compose -f infra/docker-compose.yml up -d --build knowledge-botanical
```

Tail logs for one service:

```
docker compose -f infra/docker-compose.yml logs -f agent-intake
```

3. Repo layout

frontend/	Next.js chat UI – the only thing that talks to the
gateway	
services/	
gateway/	BFF: CORS, rate limiting, proxy to orchestrator
orchestrator/	Session state machine, Tier 2 cache owner, fan-out to
agents	
agents/	intake, mapping, retrieval, safety, scoring,
explanation, reporting	
knowledge/	botanical, compound, toxicology, rules – Tier 1 lookups
email/	Verification-gated export via Resend or mock

shared/ wrapper	Python-only: Redis cache helpers + Pydantic models + LLM
seed/ loader	Starter herb/compound/symptom/rule dataset + idempotent
infra/docker-compose.yml	Local multi-service run
docs/	This guide, ARCHITECTURE.md, TECHNICAL_GUIDE.md
research docs/ authoritative for product decisions)	Original product/architecture design docs (still

Every service directory is independently buildable — its Dockerfile's build `context` is the repo root (see `infra/docker-compose.yml`), so it can `COPY shared /app/shared` etc., but nothing else about it depends on sibling services at build time.

4. Two backend stacks, one set of conventions

The backend is mid-migration (see `ARCHITECTURE.md` §7): LLM-calling services stay Python/FastAPI, everything else is moving to Java/Spring Boot. Both stacks follow the same external contract so the rest of the system never needs to know which language a given service is written in:

- Every service listens on container port **8000** and exposes `GET /healthz` returning at least `{"status": "ok"}`.
- Every service reads `REDIS_URL` from the environment (`redis://redis:6379/0` inside compose).
- Service-to-service calls always go through the plain HTTP JSON API, at the compose network alias (e.g. `http://knowledge-botanical:8000`) — never a shared library call across a service boundary, even between two Java services later on.
- Docker builds are multi-stage: a build-tool stage (`pip install`, or `mvn package`) producing an artifact, copied into a slim runtime stage. No build toolchain ships in the final image.

4a. Adding/changing a Python (FastAPI) service

1. `main.py` at the service root, `app = FastAPI(...)`, at minimum a `GET /healthz`.
2. `requirements.txt` — only list what's *not* already pulled in transitively via the shared package (`shared/pyproject.toml` already provides `pydantic`, `redis`, `httpx`, `anthropic` — don't re-list them).
3. Dockerfile pattern (copy this from any existing Python service, e.g. `services/knowledge/botanical` before its Java rewrite, or `services/agents/scoring/Dockerfile` today): `python:3.11-slim` base, `COPY shared + pip install -e /app/shared`, then the service's own requirements and source.
4. Add the service to `infra/docker-compose.yml` under the `x-backend-service` anchor (`env_file`, `REDIS_URL`, depends on `redis: service_healthy`), pick an unused host port in the 808x/809x/ 810x range already established.
5. If it needs Tier 1 data, call `await load_seed()` on startup (see any `knowledge/*` service) — it's idempotent, safe to call from multiple services' startup hooks.

4b. Migrating a service from Python to Java (the established pattern)

Follow `services/knowledge/botanical`'s layout as the template:

services/<name>/ pom.xml	Spring Boot 3 parent, spring-boot-starter-web
-----------------------------	---

```

+ spring-boot-starter-data-redis
Dockerfile      multi-stage: maven:3.9-eclipse-temurin-21 build
                 -> eclipse-temurin:21-jre runtime

src/main/java/app/rootwell/<name>/
  <Name>Application.java    @SpringBootApplication entrypoint
  <Name>Controller.java     REST endpoints, matching the old FastAPI
                           routes byte-for-byte (same paths, same
                           JSON shapes) so no caller needs to change
                           Redis access, if the service reads Tier 1 data
                           copy as-is; see §6 below
  RefCacheService.java
  CorrelationIdFilter.java
src/main/resources/
  application.yml          server.port: 8000, spring.data.redis.url:
${REDIS_URL:...}

```

Steps:

1. Read the existing `main.py` fully first — the FastAPI routes *are* the contract. Note the exact response shapes and status codes (including 404 bodies) other services depend on — grep the rest of `services/` for the old service's URL env var and endpoint paths to find every caller.
2. Prefer generic `Map<String, Object>/Jackson JsonNode` over a strict POJO for anything that's just Redis-JSON passed through — the Python side (`shared/shared/models.py`) owns the actual schema; a Java POJO that re-declares every field risks silently dropping one if the seed data changes later.
3. Use cursor-based `SCAN` (via `RedisConnection.keyCommands().scan(...)`), not `KEYS`, to enumerate `ref:*` keys — matches the Python side's `scan_iter` and avoids a blocking full-keyspace walk.
4. Update `infra/docker-compose.yml`'s block for that service to point at the new Dockerfile — nothing else in compose needs to change (port mapping, `depends_on`, env vars all stay put).
5. Rebuild just that service and run the full manual walkthrough (§5) end-to-end to confirm every other (still-Python) service that talks to it is unaffected.
6. Update `ARCHITECTURE.md` §7's status table and `CLAUDE.md`'s open items.

5. Manual verification walkthrough

```

# Health check every service
curl -s http://localhost:8082/healthz          # gateway

# Full pipeline
SID=$(curl -s -X POST http://localhost:8082/session | jq -r .session_id)
curl -s -X POST http://localhost:8082/session/$SID/message -d '{"text":"I have a
headache and trouble sleeping"}' -H 'Content-Type: application/json'
curl -s -X POST http://localhost:8082/session/$SID/advance-to-causes
curl -s -X POST http://localhost:8082/session/$SID/message -d '{"text":"work has
been stressful"}' -H 'Content-Type: application/json'
curl -s -X POST http://localhost:8082/session/$SID/analyze | jq .

# Email export (mock mode – code is printed in the email service's logs)
curl -s -X POST http://localhost:8082/session/$SID/email/request -d
'{"to":"you@example.com"}' -H 'Content-Type: application/json'
docker compose -f infra/docker-compose.yml logs email | tail -5 # grab the mock
code
curl -s -X POST http://localhost:8082/session/$SID/email/confirm -d

```

```
'{"verification_token":"<token>","code":"<code>"}' -H 'Content-Type:
application/json'

# Confirm hard purge
redis-cli -p 6380 keys "session:$SID:*" # should return empty
```

6. Observability conventions (Java services)

Every Java service registers `CorrelationIdFilter`: it reads (or mints) an `X-Correlation-Id` header, puts it in SLF4J's MDC for the duration of the request, and echoes it back in the response header.

`application.yml`'s `logging.pattern.console` includes `%X{correlationId}` so every log line for a request chain — including across services, once the header is forwarded on outbound calls — carries the same id. This is the minimum viable version of the logs/metrics/traces observability model; there's no metrics/tracing backend wired up yet (no Prometheus/Grafana, no distributed tracing collector) — see `docs/TECHNICAL_GUIDE.md` for what a next step would look like and why it hasn't been added yet (would require an actual operational deployment to be worth the cost, per Phase 2 scope in `CLAUDE.md`).

7. Common pitfalls

- **Port conflicts on this dev machine specifically:** gateway is on host **8082** (not 8080), Redis is on host **6380** (not 6379) — see `CLAUDE.md`'s "Known environment quirks." These are host-mapping-only; container-to-container calls are unaffected.
- **Mock mode isn't an error state.** If `docker compose logs agent-intake` shows `mock_mode: true` in a `/healthz` response, that's expected without `ANTHROPIC_API_KEY` set — it's a deliberate runtime mode, not something to "fix."
- **Don't add a database.** If a new requirement seems to need one, revisit `ARCHITECTURE.md` §3 first — the no-DB constraint is a decision, not an oversight, and most "I need a query" instincts are solved by the existing Tier 1/Tier 2 Redis key patterns instead.
- **Seed data flag:** every herb record carries `curation_status: "starter_dataset_unreviewed"`. Don't strip this anywhere in the pipeline — it's meant to reach the end user.